CVM++

Stack-Based Virtual Machine and Custom Compiler

Project Report

Team Members

- Ram Patidar
- Pranjal Singh
- Nikunj Jindal

---

1. Project Overview

CVM++ is a complete end-to-end language implementation written in C++17. It implements a small custom scripting language that is compiled into bytecode and executed by a stack-based virtual machine.

The goal of this project is to understand how modern programming languages work internally by building the core stages from scratch:

- Lexical analysis
- Parsing
- Abstract Syntax Tree generation
- Bytecode compilation
- Stack-based virtual machine execution

The language supports integers, booleans, variables, assignment, arithmetic, comparisons, conditional branching, loops, input, and print output. The implementation also includes debug modes to display the generated AST and bytecode.

The full pipeline is:

```
Source Code -> Lexer -> Tokens -> Parser -> AST -> Compiler -> Bytecode -> VM -> Output
```

---

2. Problem Statement

Most developers use high-level languages without seeing how raw source text becomes executable behavior. CVM++ addresses this by implementing a simplified but complete compiler and runtime pipeline.

The project demonstrates:

- How source code is broken into tokens
- How tokens are arranged into meaningful syntax
- How syntax is represented as an AST
- How AST nodes are converted into bytecode instructions
- How a virtual machine executes those instructions using a stack

This makes CVM++ a learning-oriented compiler and virtual machine project.

---

3. System Architecture

CVM++ follows a sequential pipeline. Each stage consumes the output of the previous stage and produces a structured result for the next stage.

```
script.cvm
    |
    v
Lexer
    |
    v
Token Stream
    |
    v
Parser
    |
    v
Abstract Syntax Tree
    |
    v
Compiler
    |
    v
Bytecode Chunk
    |
    v
Stack-Based VM
    |
    v
Program Output
```

Component Summary

Component	Files	Responsibility
Lexer	`src/lexer/lexer.h`, `src/lexer/lexer.cpp`	Converts raw source code into tokens
Parser	`src/parser/parser.h`, `src/parser/parser.cpp`	Converts tokens into an AST
AST	`src/parser/ast.h`	Represents expressions and statements
Compiler	`src/compiler/compiler.h`, `src/compiler/compiler.cpp`	Emits bytecode from AST nodes
Bytecode Chunk / ISA	`src/compiler/chunk.h`, `src/compiler/chunk.cpp`	Stores instructions, constants, and disassembly logic
Virtual Machine	`src/vm/vm.h`, `src/vm/vm.cpp`	Executes bytecode using a stack
CLI Runner	`src/main.cpp`	Connects lexer, parser, compiler, and VM

---

4. Language Features

CVM++ intentionally keeps the language simple and focused on compiler and VM concepts.

Supported features:

- Integer literals
- Boolean literals: `true`, `false`
- Variable declaration using `let`
- Variable assignment
- Arithmetic operators: `+`, `-`, `*`, `/`

- Comparison operators: ==, <, >
- Unary minus
- `if/else` control flow
- `while` loops
- `input` keyword
- `print` keyword
- Block scope
- Variable shadowing

Example:

```
let x = 1
if x == 1 {
  let x = 2
  print x
}
print x
```

Output:

```
2
1
```

The inner `x` shadows the outer `x` only inside the block.

---

5. Lexer

The lexer performs a linear scan over the source code and converts characters into tokens.

It recognizes:

- Number literals
- Identifiers
- Keywords: `let`, `print`, `input`, `if`, `else`, `while`, `true`, `false`
- Arithmetic operators: `+`, `-`, `*`, `/`
- Comparison operators: ==, <, >
- Assignment operator: =
- Block delimiters: `{`, `}`

Example source:

```
let x = 10 + 5
```

Token stream:

```
LET IDENTIFIER ASSIGN NUMBER PLUS NUMBER EOF
```

Unknown characters are reported as lexical errors. For example, semicolons are not part of the current language, so this gives an error:

```
let x = 1;
```

---

6. Parser and AST

The parser is implemented using recursive descent parsing. It consumes tokens from the lexer and builds an Abstract Syntax Tree.

The parser separates statements and expressions.

Statement nodes include:

- `LetStmt`
- `AssignStmt`
- `PrintStmt`
- `InputStmt`
- `IfStmt`
- `WhileStmt`

Expression nodes include:

- `NumberLiteral`
- `BoolLiteral`
- `IdentExpr`
- `BinaryExpr`

Operator precedence is handled using separate parsing functions:

```
parseExpression
    -> parseAddSub
    -> parseMulDiv
    -> parseUnary
    -> parsePrimary
```

This ensures multiplication and division bind more tightly than addition and subtraction.

Example:

```
let total = 10 + 5 * 2
```

AST structure:

```
LetStmt(total)
  BinaryExpr(+)
    NumberLiteral(10)
    BinaryExpr(*)
      NumberLiteral(5)
      NumberLiteral(2)
```

The AST uses `std::unique_ptr` for ownership, which avoids manual memory management and memory leaks.

---

7. Scope Handling

CVM++ supports block scoping. Variables declared inside a block do not leak outside the block.

Example:

```
let x = 1
if x == 1 {
  let y = 2
  print y
}
print y
```

The final `print y` is rejected because `y` exists only inside the `if` block.

Both the parser and compiler maintain scope information:

- Parser scope checks prevent undeclared variable usage.
- Compiler scope resolution maps variable names to bytecode slots.
- Shadowed variables receive separate slots.

---

8. Compiler

The compiler walks the AST and emits bytecode instructions into a `Chunk`.

The compiler uses the Visitor pattern:

```
AST Node -> accept(visitor) -> Compiler::visit(...)
```

This keeps AST classes clean and allows the compiler to define behavior for each node type.

Example source:

```
print 10 + 5
```

Generated bytecode:

```
PUSH_INT 10
PUSH_INT 5
ADD
PRINT
HALT
```

Control flow is implemented using jump instructions:

- `JUMP`
- `JUMP_IF_FALSE`

For `if/else`, the compiler emits placeholder jumps and patches them after the target location is known.

For `while`, the compiler records the loop start and emits a backward jump at the end of the loop body.

---

9. Instruction Set Architecture

The ISA is defined in `src/compiler/chunk.h`.

Opcode	Operand	Description
`PUSH_INT`	constant index	Push integer constant onto stack
`PUSH_BOOL`	`0` or `1`	Push boolean value onto stack
`LOAD_VAR`	variable slot	Push variable value
`STORE_VAR`	variable slot	Store value into existing variable
`DEFINE_VAR`	variable slot	Define variable from stack value
`ADD`	none	Integer addition
`SUB`	none	Integer subtraction
`MUL`	none	Integer multiplication
`DIV`	none	Integer division

`EQ`	none	Equality comparison
`LT`	none	Less-than comparison
`GT`	none	Greater-than comparison
`NEGATE`	none	Integer negation
`JUMP`	instruction index	Unconditional jump
`JUMP_IF_FALSE`	instruction index	Conditional jump
`PRINT`	none	Print top stack value
`INPUT`	variable slot	Read integer input into variable
`HALT`	none	Stop execution

Bytecode is stored as a vector of `Instruction` objects. Each instruction contains:

- an opcode
- an optional integer operand

The bytecode disassembler prints constants, variable slots, and instructions in readable form.

---

10. Virtual Machine

The VM executes bytecode using a fetch-decode-execute loop.

Internal VM state:

- `ip`: instruction pointer
- `stack`: operand stack
- `vars`: variable slots

Execution loop:

```
while not halted:
    fetch instruction at ip
    increment ip
    execute instruction
```

Arithmetic instructions pop operands from the stack and push the result back.

Example:

```
PUSH_INT 10
PUSH_INT 5
ADD
PRINT
```

Stack behavior:

```
[]
[10]
[10, 5]
[15]
print 15
```

Runtime values are represented using:

```
std::variant<int, bool>
```

This provides type-safe handling for integers and booleans.

---

11. Error Handling

CVM++ reports errors across different stages.

Lexical Error

Unsupported characters are rejected.

```
let x = 1;
```

Output:

```
Lex error: unknown character ';'
```

Parse Error

Undeclared variables are rejected.

```
print x
```

Output:

```
Parse error: use of undeclared variable 'x'
```

Runtime Error

Division by zero is caught in the VM.

```
let x = 10 / 0
print x
```

Output:

```
VM: division by zero
```

Runtime type errors are also handled, such as trying to add a boolean and an integer.

---

12. Build and Run Instructions

From the project root:

```
g++ -std=c++17 -Wall -Wextra -Werror -Isrc src/main.cpp src/lexer/lexer.cpp src/parser/parser.cpp src/compiler/c
```

Run a program:

```
.\cvm.exe examples\full_demo.cvm
```

Show AST:

```
.\cvm.exe --ast examples\full_demo.cvm
```

Show bytecode:

```
.\cvm.exe --bytecode examples\full_demo.cvm
```

Show AST and bytecode without executing:

```
.\cvm.exe --ast --bytecode --no-run examples\full_demo.cvm
```

Run all tests:

```
powershell -ExecutionPolicy Bypass -File tests\run_all_tests.ps1
```

13. Testing

The project includes parser, compiler, and VM tests.

Test categories:

- Valid parser programs
- Invalid parser programs
- Bytecode generation
- Scoped variable shadowing
- VM execution output
- Division by zero
- Invalid input
- Runtime type errors
- Unsupported syntax

Expected result:

```
All parser tests passed.
All compiler tests passed.
All VM tests passed.
All CVM tests passed.
```

14. Design Decisions

Stack-Based VM

CVM++ uses a stack-based VM because it is simpler and easier to understand than a register-based VM. Arithmetic operations do not need explicit operand fields because operands are taken from the top of the stack.

Recursive Descent Parser

Recursive descent was chosen because the grammar is small and easy to express as functions. It also makes operator precedence clear.

Visitor Pattern

The compiler uses the Visitor pattern to walk AST nodes. Each AST node calls the correct `visit` function on the compiler.

Compile-Time Variable Resolution

Variable names are resolved to integer slots during compilation. The VM uses slot indices instead of variable names, which keeps runtime execution simple.

Smart Pointers

AST nodes are owned using `std::unique_ptr`, avoiding manual `new` and `delete`.

15. Known Limitations

The project intentionally keeps the language small. It does not currently support:

- Strings
- Functions
- Arrays
- Classes
- Comments
- Semicolons
- Parentheses
- <=, >=, !=
- Floating point numbers

These are possible future extensions.

---

16. Conclusion

CVM++ successfully implements the major components of a small programming language:

- lexer
- parser
- AST
- bytecode compiler
- instruction set architecture
- stack-based virtual machine
- debug output
- sample programs
- automated tests

The project satisfies the Coding Club CVM++ scope and demonstrates how high-level source code can be translated into bytecode and executed by a custom virtual machine.

---

17. References

- Robert Nystrom, *Crafting Interpreters*
- Stack machine concept
- C++17 standard library features: `std::vector`, `std::variant`, `std::unique_ptr`